

Assignment 3: To develop a distributed system, to find the minimum number of N elements in an array by distributing N/n elements to n number of processors MPI or OpenMP. Demonstrate by displaying the intermediate minimum element calculated at different processors.

```
#include <stdio.h>
#include "mpi.h"

int main(int argc, char* argv[])
{
    int rank, size;
    int num[20]; // N = 20

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Initialize array
    for(int i = 0; i < 20; i++)
        num[i] = i + 1;

    if(rank == 0)
    {
        int s[4];
        printf("Distribution at rank %d\n", rank);

        // Send N/n elements (20/4 = 5)
        for(int i = 1; i < 4; i++)
            MPI_Send(&num[i * 5], 5, MPI_INT, i, 1, MPI_COMM_WORLD);

        int local_max = num[0];

        for(int i = 1; i < 5; i++)
        {
            if(num[i] > local_max)
                local_max = num[i];
        }

        // Receive from other processes
        for(int i = 1; i < 4; i++)
        {
            MPI_Recv(&s[i], 1, MPI_INT, i, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        }

        printf("Local max at rank %d is %d\n", rank, local_max);

        int final_max = local_max;

        for(int i = 1; i < 4; i++)
        {
            if(s[i] > final_max)
                final_max = s[i];
        }

        printf("Final Maximum = %d\n\n", final_max);
    }
    else
    {
        int k[5];
        MPI_Recv(k, 5, MPI_INT, 0, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        int local_max = k[0];

        for(int i = 1; i < 5; i++)
        {
            if(k[i] > local_max)
                local_max = k[i];
        }
    }
}
```

```
    printf("Local max at rank %d is %d\n", rank, local_max);

    MPI_Send(&local_max, 1, MPI_INT, 0, 1, MPI_COMM_WORLD);
}

MPI_Finalize();
return 0;
}
```

Output:

```
Distribution at rank 0
Local max at rank 1 is 10
Local max at rank 2 is 15
Local max at rank 3 is 20
Local max at rank 0 is 5
Final Maximum = 20
```

Assignment 1: Implement multi-threaded client/server Process communication using RMI.

Find sum of squares of 2 numbers

Client.java

```
import java.rmi.*;
import java.util.Scanner;

public class Client {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            String serverUrl = "rmi://localhost/Server";
            ServerInterface serverIntf = (ServerInterface) Naming.lookup(serverUrl);

            System.out.print("Enter first number: ");
            double a = sc.nextDouble();

            System.out.print("Enter second number: ");
            double b = sc.nextDouble();

            double result = serverIntf.addSquares(a, b);

            System.out.println("Result (a^2 + b^2) = " + result);

        } catch (Exception e) {
            System.out.println("Exception at Client: " + e);
        }
    }
}
```

Server.java

```
import java.rmi.*;

public class Server {
    public static void main(String[] args) {
        try {
            ServerImplementation serverImpl = new ServerImplementation();
            Naming.rebind("Server", serverImpl);
            System.out.println("Server is Ready...");
        } catch (Exception e) {
            System.out.println("Exception at Server: " + e);
        }
    }
}
```

ServerInterface.java

```
import java.rmi.*;

interface ServerInterface extends Remote {
    public double addSquares(double a, double b) throws RemoteException;
}
```

ServerImplementation

```
import java.rmi.*;
import java.rmi.server.*;

public class ServerImplementation extends UnicastRemoteObject implements ServerInterface {

    public ServerImplementation() throws RemoteException { }

    public double addSquares(double a, double b) throws RemoteException {
        return (a * a) + (b * b);
    }
}
```

OUTPUT

Terminal 1

```
C:\Users\chet0\OneDrive\Desktop\LP V\A1>rmiregistry
```

Terminal 2

```
C:\Users\chet0\OneDrive\Desktop\LP V\A1>java Server
```

Server is Ready...

Terminal 3

```
C:\Users\chet0\OneDrive\Desktop\LP V\A1>java Client
```

Enter first number: 86

Enter second number: 2

Sum of Squares of 86.0 and 2.0 is: 7400.0

**Assignment 2 : Develop any distributed application using CORBA to demonstrate object brokering.
(Calculator Operations)**

Client.java

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import CalcApp.*;
import CalcApp.CalcPackage.DivisionByZero;
import org.omg.CosNaming.*;
import org.omg.CosNaming.NamingContextPackage.*;
import org.omg.CORBA.*;
import static java.lang.System.out;

public class CalcClient {
    static Calc calcImpl;
    static BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

    public static void main(String args[]) {

        try {
            ORB orb = ORB.init(args, null);
            org.omg.CORBA.Object objRef = orb.resolve_initial_references("NameService");
            NamingContextExt ncRef = NamingContextExtHelper.narrow(objRef);
            String name = "Calc";
            calcImpl = CalcHelper.narrow(ncRef.resolve_str(name));
            while (true) {
                out.println("1. Sum");
                out.println("2. Sub");
                out.println("3. Mul");
                out.println("4. Div");
                out.println("5. exit");
                out.println("--");
                out.println("choice: ");

                try {
                    String opt = br.readLine();
                    if (opt.equals("5")) {
                        break;
                    } else if (opt.equals("1")) {
                        out.println("a+b= " + calcImpl.sum(getFloat("a"), getFloat("b")));
                    } else if (opt.equals("2")) {
                        out.println("a-b= " + calcImpl.sub(getFloat("a"), getFloat("b")));
                    } else if (opt.equals("3")) {
                        out.println("a*b= " + calcImpl.mul(getFloat("a"), getFloat("b")));
                    } else if (opt.equals("4")) {
                        try {
                            out.println("a/b= " + calcImpl.div(getFloat("a"), getFloat("b")));
                        } catch (DivisionByZero de) {
                            out.println("Division by zero!!!");
                        }
                    }
                } catch (Exception e) {
                    out.println("====");
                    out.println("Error with numbers");
                    out.println("====");
                }
                out.println("");
            }
            //calcImpl.shutdown();
        } catch (Exception e) {
            System.out.println("ERROR : " + e);
            e.printStackTrace(System.out);
        }
    }
}
```

```

static float getFloat(String number) throws Exception {
    out.print(number + ": ");
    return Float.parseFloat(br.readLine());
}
}

```

Server.java

```

import CalcApp.*;
import CalcApp.CalcPackage.DivisionByZero;

import org.omg.CosNaming.*;
import org.omg.CosNaming.NamingContextPackage.*;
import org.omg.CORBA.*;
import org.omg.PortableServer.*;

import java.util.Properties;

class CalcImpl extends CalcPOA {

    @Override
    public float sum(float a, float b) {
        return a + b;
    }
    @Override
    public float div(float a, float b) throws DivisionByZero {
        if (b == 0) {
            throw new CalcApp.CalcPackage.DivisionByZero();
        } else {
            return a / b;
        }
    }
    @Override
    public float mul(float a, float b) {
        return a * b;
    }
    @Override
    public float sub(float a, float b) {
        return a - b;
    }
    private ORB orb;
    public void setORB(ORB orb_val) {
        orb = orb_val;
    }
}

public class CalcServer {
    public static void main(String args[]) {
        try {
            ORB orb = ORB.init(args, null);
            POA rootpoa = POAHelper.narrow(orb.resolve_initial_references("RootPOA"));
            rootpoa.the_POAManager().activate();
            CalcImpl helloImpl = new CalcImpl();
            helloImpl.setORB(orb);

            org.omg.CORBA.Object ref = rootpoa.servant_to_reference(helloImpl);
            Calc href = CalcHelper.narrow(ref);

            org.omg.CORBA.Object objRef = orb.resolve_initial_references("NameService");

            NamingContextExt ncRef = NamingContextExtHelper.narrow(objRef);

            // bind the Object Reference in Naming
            String name = "Calc";
            NameComponent path[] = ncRef.to_name(name);
            ncRef.rebind(path, href);

            System.out.println("Ready..");
        }
    }
}

```

```

        // wait for invocations from clients
        orb.run();
    } catch (Exception e) {
        System.err.println("ERROR: " + e);
        e.printStackTrace(System.out);
    }

    System.out.println("Exiting ...");

}
}

```

Output

Terminal 1

```

C:\Users\chet8>cd C:\Users\chete\OneDrive\Desktop\LP V\A2
C:\Users\chete\OneDrive\Desktop\LP V\A2>java CalcServer -ORBInitialPort 1058 -ORBInitialHost localhost
Ready..

```

Terminal 2

```

C:\Users\chet@>cd C:\Users\chet@\OneDrive\Desktop\LP V\A2
C:\Users\chet@\OneDrive\Desktop\LP V\A2>java CalcClient -ORBInitialPort 105

```

```

1. Sum
2. Sub
3. Mul
4. Div
5. exit
choice:3
a: 569
b: 87
a+b= 49503.8

```

```

1. Sum
2. Sub
3. Mul
4. Div
5. exit
choice:2
a: 89
b: 47
a-b= 42.8

```

```

1 Sum
2. Sub
3. Mul
4. Div
5. exit
choice: 4
a: 360
b: 6
a/b= 60.0

```

```

1. Sum
2. Sub
3. Mul
4. Div
5. exit
choice:3
a: 360
b: 6
a*b = 216

```

Assignment 4: Implement Berkeley algorithm for clock synchronization.

```
import java.util.*;
class Berkeley {
    public static void synchronize(List<Integer> clocks) {
        int n = clocks.size();
        int sum = 0;
        for (int time : clocks) {
            sum += time;
        }
        int avg = sum / n;
        System.out.println("\nAverage Time: " + avg);
        // Step 3: Calculate adjustments and apply
        System.out.println("\nAdjustments:");
        for (int i = 0; i < n; i++) {
            int current = clocks.get(i);
            int adjustment = avg - current;
            System.out.println("Process " + i + " adjustment = " + adjustment);
            clocks.set(i, current + adjustment);
        }
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of clocks: ");
        int n = sc.nextInt();
        List<Integer> clocks = new ArrayList<>();
        System.out.println("Enter clock times:");
        for (int i = 0; i < n; i++) {
            clocks.add(sc.nextInt());
        }
        System.out.println("\nInitial Clock Times:");
        for (int t : clocks) {
            System.out.println(t);
        }
        // Apply Berkeley Algorithm
        synchronize(clocks);
        System.out.println("\nSynchronized Clock Times:");
        for (int t : clocks) {
            System.out.println(t);
        }
    }
}
```

Output

C:\Users\chet0\OneDrive\Desktop\LP V\A4>java Berkeley

Enter number of clocks: 3

Enter clock times:

200

50

200

Initial Clock Times:

200

50

200

Average Time: 150

Adjustments:

Process 0 adjustment = -50

Process 1 adjustment = 100

Process 2 adjustment = -50

Synchronized Clock Times:

150

150

150

Assignment 5 : Implement token ring based mutual exclusion algorithm.

```
import java.util.*;
class tokenring {
    public static void main(String args[]) {
        Scanner scan = new Scanner(System.in);
        System.out.println("Enter the num of nodes:");
        int n = scan.nextInt();
        int token = 0;
        int ch;
        System.out.println("Ring:");
        for (int i = 0; i < n; i++) {
            System.out.print(" " + i);
        }
        System.out.println(" " + 0);
        do {
            System.out.println("\nCurrent Token at: " + token);
            System.out.println("Enter sender:");
            int s = scan.nextInt();
            System.out.println("Enter receiver:");
            int r = scan.nextInt();
            System.out.println("Enter Data:");
            int data = scan.nextInt();
            System.out.print("Token passing: ");
            while (token != s) {
                System.out.print(token + " -> ");
                token = (token + 1) % n;
            }
            System.out.println(s);
            System.out.println("Sender " + s + " sending data: " + data);
            // Data forwarding
            int temp = s;
            while (temp != r) {
                temp = (temp + 1) % n;
                System.out.println("Data " + data + " forwarded by " + temp);
            }
            System.out.println("Receiver " + r + " received data: " + data);
            token = (r + 1) % n;
            System.out.print("\nSend again? (1-Yes / 0-No): ");
            ch = scan.nextInt();
        } while (ch == 1);
        scan.close();
    }
}
```

Output

C:\Users\chet0\OneDrive\Desktop\LP V\A5>java tokenring

Enter the num of nodes:

4

Ring:

0 1 2 3 0

Current Token at: 0

Enter sender:

2

Enter receiver:

1

Enter Data:

55

Token passing: 0 -> 1 -> 2

Sender 2 sending data: 55

Data 55 forwarded by 3

Data 55 forwarded by 0

Data 55 forwarded by 1

Receiver 1 received data: 55

Send again? (1-Yes / 0-No): 0

Assignment 6: Implement Bully and Ring algorithm for leader election.

Ring.java

```
import java.util.Scanner;

public class Ring {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.println("Enter the number of processes: ");
        int num = in.nextInt();

        Rr[] proc = new Rr[num];
        for (int i = 0; i < num; i++) {
            proc[i] = new Rr();
            proc[i].index = i;
            System.out.println("Enter the ID of process " + (i + 1) + ": ");
            proc[i].id = in.nextInt();
            proc[i].state = "active";
            proc[i].f = 0;
        }
        for (int i = 0; i < num - 1; i++) {
            for (int j = 0; j < num - 1; j++) {
                if (proc[j].id > proc[j + 1].id) {
                    Rr temp = proc[j];
                    proc[j] = proc[j + 1];
                    proc[j + 1] = temp;
                }
            }
        }
        for (int i = 0; i < num; i++) {
            System.out.print "[" + i + " " + proc[i].id + " ";
        }
        proc[num - 1].state = "inactive";
        System.out.println("\nProcess " + proc[num - 1].id + " selected as coordinator");
        while (true) {
            System.out.println("\n1. Election\n2. Quit");
            int ch = in.nextInt();

            // Reset flags
            for (int i = 0; i < num; i++) {
                proc[i].f = 0;
            }

            switch (ch) {
                case 1:
                    System.out.println("Enter the process number that initializes the election: ");
                    int init = in.nextInt();
                    int temp2 = init;
                    int temp1 = init + 1;
                    int i = 0;

                    while (temp2 != temp1) {
                        if (temp1 == num) {
                            temp1 = 0;
                        }
                        if ("active".equals(proc[temp1].state) && proc[temp1].f == 0) {
                            System.out.println("Process " + proc[init].id + " sends a message to Process " + proc[temp1].id);
                            proc[temp1].f = 1;
                            init = temp1;
                            i++;
                        }
                        temp1++;
                    }

                    System.out.println("Process " + proc[init].id + " sends a message to Process " + proc[temp1].id);
                    int max = -1;
```

```

        int m = -1;
for (int j = 0; j < num; j++) {
    if (proc[j].id > m) {
        m = proc[j].id;
    }
}

        System.out.println("Process " + m + " selected as coordinator");
        for (int k = 0; k < num; k++) {
            if (proc[k].id == m) {
                proc[k].state = "inactive";
            }
        }
        break;
case 2:
    System.out.println("Program terminated.");
    in.close();
    return;
default:
    System.out.println("Invalid response.");
    break;
    }
    }
}

class Rr {
    public int index; // To store the index of the process
    public int id; // To store the ID/name of the process
    public int f;
    public String state; // Indicates whether the process is active or inactive
}

```

Output

C:\Users\chet0\OneDrive\Desktop\LP V\A6>java Ring

Enter the number of processes:

4

Enter the ID of process 1:

2

Enter the ID of process 2:

3

Enter the ID of process 3:

4

Enter the ID of process 4:

5

[0] 2 [1] 3 [2] 4 [3] 5

Process 5 selected as coordinator

1. Election

2. Quit

1

Enter the process number that initializes the election:

3

Process 5 sends a message to Process 2

Process 2 sends a message to Process 3

Process 3 sends a message to Process 4

Process 4 sends a message to Process 5

Process 5 selected as coordinator

1. Election

2. Quit

Bully.java

```
import java.util.Scanner;
```

```
public class Bully {
    static boolean[] state = new boolean[5];
    public static int coordinator = 4;
    public static void getStatus() {
        System.out.println("\n+-----Current System State-----+");
        for (int i = 0; i < state.length; i++) {
            System.out.println(" P" + (i + 1) + ":\t" + (state[i] ? "UP" : "DOWN") +
                (coordinator == i ? "\t<-- COORDINATOR\t" : "\t\t\t"));
        }
        System.out.println("+-----+");
    }
    public static void up(int up) {
        if (state[up - 1]) {
            System.out.println("Process " + up + " is already up");
        } else {
            state[up - 1] = true;
            System.out.println("-----Process " + up + " held election-----");
            for (int i = up; i < state.length; ++i) {
                System.out.println("Election message sent from process " + up + " to process " + (i + 1));
            }
            for (int i = state.length - 1; i >= 0; --i) {
                if (state[i]) {
                    coordinator = i;
                    break;
                }
            }
        }
    }
    public static void down(int down) {
        if (!state[down - 1]) {
            System.out.println("Process " + down + " is already down.");
        } else {
            state[down - 1] = false;
            if (coordinator == down - 1) {
                setCoordinator();
            }
        }
    }
    public static void mess(int mess) {
        if (state[mess - 1]) {
            if (state[coordinator]) {
                System.out.println("Message Sent: Coordinator is alive");
            } else {
                System.out.println("Coordinator is down");
                System.out.println("Process " + mess + " initiated election");
                for (int i = mess; i < state.length; ++i) {
                    System.out.println("Election sent from process " + mess + " to process " + (i + 1));
                }
                setCoordinator();
            }
        } else {
            System.out.println("Process " + mess + " is down");
        }
    }
    public static void setCoordinator() {
        for (int i = state.length - 1; i >= 0; i--) {
            if (state[i]) {
                coordinator = i;
                break;
            }
        }
    }
}
```

```

    }

    public static void main(String[] args) {
        int choice;
        Scanner sc = new Scanner(System.in);
        for (int i = 0; i < state.length; ++i) {
            state[i] = true;
        }
        getStatus();
        do {
            System.out.println("+.....MENU.....+");
            System.out.println("1. Activate a process.");
            System.out.println("2. Deactivate a process.");
            System.out.println("3. Send a message.");
            System.out.println("4. Exit.");
            System.out.println("+.....+");
            choice = sc.nextInt();
            switch (choice) {
                case 1: {
                    System.out.println("Activate process:");
                    int up = sc.nextInt();
                    if (up == 5) {
                        System.out.println("Process 5 is the coordinator");
                        state[4] = true;
                        coordinator = 4;
                        break;
                    }
                    up(up);
                    break;
                }
                case 2: {
                    System.out.println("Deactivate process:");
                    int down = sc.nextInt();
                    down(down);
                    break;
                }
                case 3: {
                    System.out.println("Send message from process:");
                    int mess = sc.nextInt();
                    mess(mess);
                    break;
                }
            }
            getStatus();
        } while (choice != 4);
        sc.close();
    }
}

```

Output

C:\Users\chet0\OneDrive\Desktop\LP V\A6>java Bully

```

+-----Current System State-----+
| P1:  UP          |
| P2:  UP          |
| P3:  UP          |
| P4:  UP          |
| P5:  UP    <-- COORDINATOR |
+-----+
+.....MENU.....+
1. Activate a process.
2. Deactivate a process.
3. Send a message.
4. Exit.
+.....+
2
Deactivate process:

```

```

+-----Current System State-----+
| P1:  UP          |
| P2:  UP          |
| P3:  UP          |
| P4:  UP    <-- COORDINATOR |
| P5:  DOWN        |
+-----+
+.....MENU.....+
1. Activate a process.
2. Deactivate a process.
3. Send a message.
4. Exit.
+.....+

```

3

Send message from process:

5

Process 5 is down

```

+-----Current System State-----+
| P1:  UP          |
| P2:  UP          |
| P3:  UP          |
| P4:  UP    <-- COORDINATOR |
| P5:  DOWN        |
+-----+
+.....MENU.....+
1. Activate a process.
2. Deactivate a process.
3. Send a message.
4. Exit.
+.....+

```

1

Activate process:

5

Process 5 is the coordinator

```

+-----Current System State-----+
| P1:  UP          |
| P2:  UP          |
| P3:  UP          |
| P4:  UP          |
| P5:  UP    <-- COORDINATOR |
+-----+
+.....MENU.....+
1. Activate a process.
2. Deactivate a process.
3. Send a message.
4. Exit.
+.....+

```

4

```

+-----Current System State-----+
| P1:  UP          |
| P2:  UP          |
| P3:  UP          |
| P4:  UP          |
| P5:  UP    <-- COORDINATOR |
+-----+

```

Assignment No. 7: Create a simple web service and write any distributed application to consume the web service. – Temperature Converter

Server.py

```
import requests
def convert_temperature():
    value = float(input("Enter temperature value: "))
    unit = input("Enter unit (C/F): ").upper()

    url = 'http://localhost:5000/convert'
    data = {
        'value': value,
        'unit': unit
    }

    response = requests.post(url, json=data)
    result = response.json()

    if 'error' in result:
        print(result['error'])
    else:
        print(f"\n {result['original']}°{result['unit']} = "
              f"{result['converted_value']}°{result['converted_to']}")
convert_temperature()
```

Client.py

```
from flask import Flask, request, jsonify
app = Flask(__name__)
@app.route('/convert', methods=['POST'])
def convert_temp():
    data = request.get_json()
    value = float(data['value'])
    unit = data['unit'].upper()
    if unit == 'C':
        result = (value * 9/5) + 32
        converted_to = 'F'
    elif unit == 'F':
        result = (value - 32) * 5/9
        converted_to = 'C'
    else:
        return jsonify({'error': 'Invalid unit. Use C or F'}), 400
    return jsonify({
        'original': value,
        'unit': unit,
        'converted_value': round(result, 2),
        'converted_to': converted_to
    })
if __name__ == '__main__':
    app.run()
```

Output

Terminal 1

```
C:\Users\chet0\OneDrive\Desktop\LP V\A7>python server.py
* Serving Flask app 'server'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI
server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [24/Apr/2026 13:45:52] "POST /convert HTTP/1.1" 200 -
```

Terminal 2

```
C:\Users\chet0\OneDrive\Desktop\LP V\A7>python app.py
Enter temperature value: 37
Enter unit (C/F): c
Result:37.0°C = 98.6°F
```